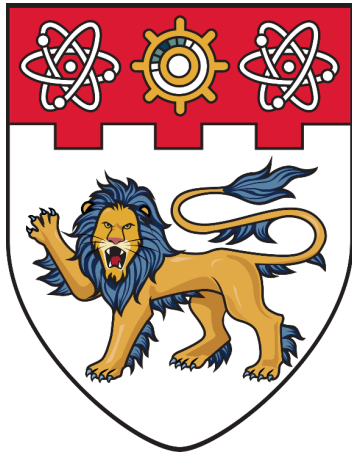




**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

SC4000 Machine Learning

Project Title: Elo Merchant Category Recommendation

Members:

WU JUN HUI (U2021857H); distribution of work: 20%

KIM SEO JIN (U2122248G); distribution of work: 20%

TEO SHAO QI (U2222637B); distribution of work: 20%

BHATI NANCY (U2222368G); distribution of work: 20%

BANKATA MISHRA TRISHA (U2222358J); distribution of work: 20%

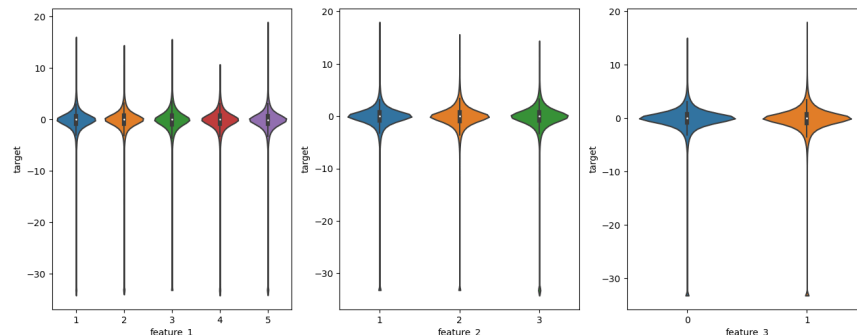
1. Introduction

This project looks into various machine learning models to be applied in the Kaggle competition: **ELO Merchant Category Recommendation**, where we predict customer loyalty using limited datasets. Information such as 1) card features, 2) historical transactions (three months' worth of transactions before 1st February 2018), 3) new transactions (transactions that had not been made at merchants specified in historical_transactions for a period of two months) and 4) merchant data, were provided to help us predict the loyalty score for each card. February 2018 is inferred to be the month ELO Merchant began their recommendation according to the transactional data. We've applied machine learning models learned and not covered in class: Decision Tree, Random Forest, XGBoost, LightGBM and CatBoost, and explored ensemble learning (stacking, with neural network) to improve our model's overall performance. We also referenced existing Kaggle solutions, including 7th-place (RMSE 3.64108) and 10th-place (RMSE 3.64604) implementations.

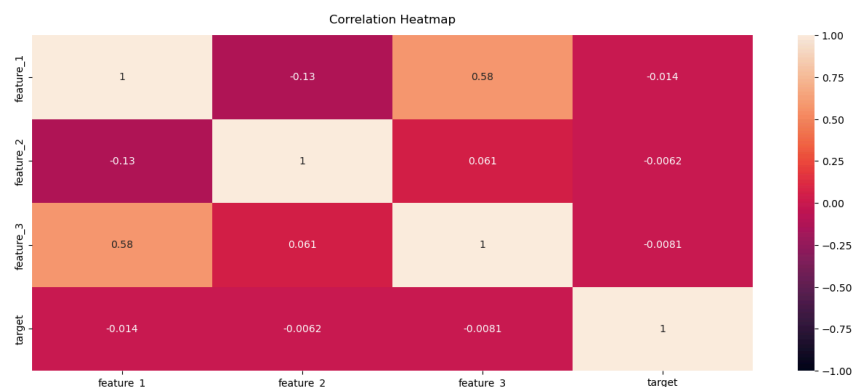
2. Exploratory Data Analysis

Data definitions are found [here](#).

We began by examining information specific to each card from train.csv. Here, the correlation of features against loyalty score (target) is analysed.

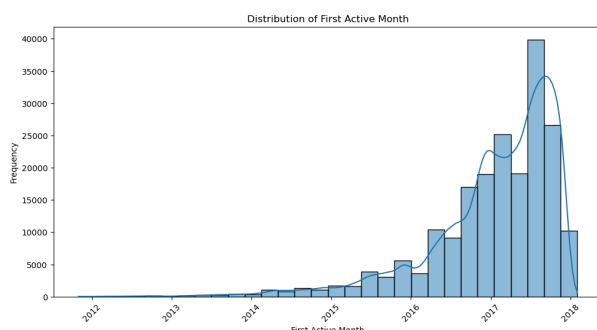


Violin Plot showing spread of feature_1, feature_2 and feature_3 from train.csv



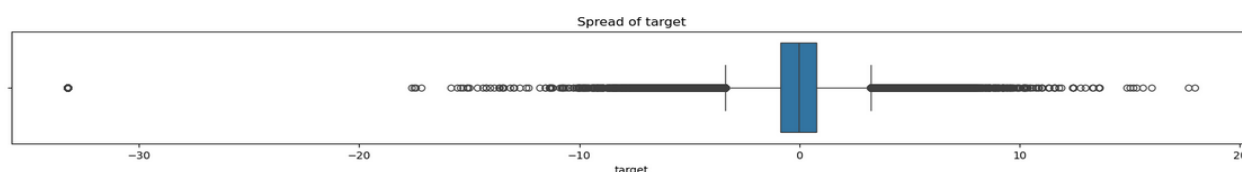
Heatmap of correlation between features in train.csv

From the violin plots we saw that feature_1, feature_2, and feature_3 are evenly spread across the target values, making the features less ideal in predicting loyalty scores. This was further confirmed by the heatmap, where we saw weak correlation scores between the three features and target.



Distribution of the months of first active transactions

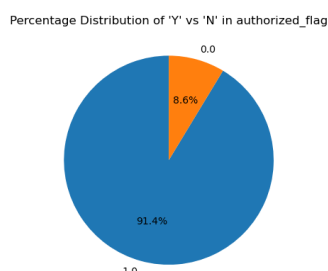
From the distribution, most cardholders made their first transactions between 2017 and 2018. While not definitive, it may suggest a surge in sign-ups during the same period.



Distribution of 'target'

From the distribution above there seemed to be some outliers at targets below -30. See challenges as we describe the problems faced with these outliers and how we handle them.

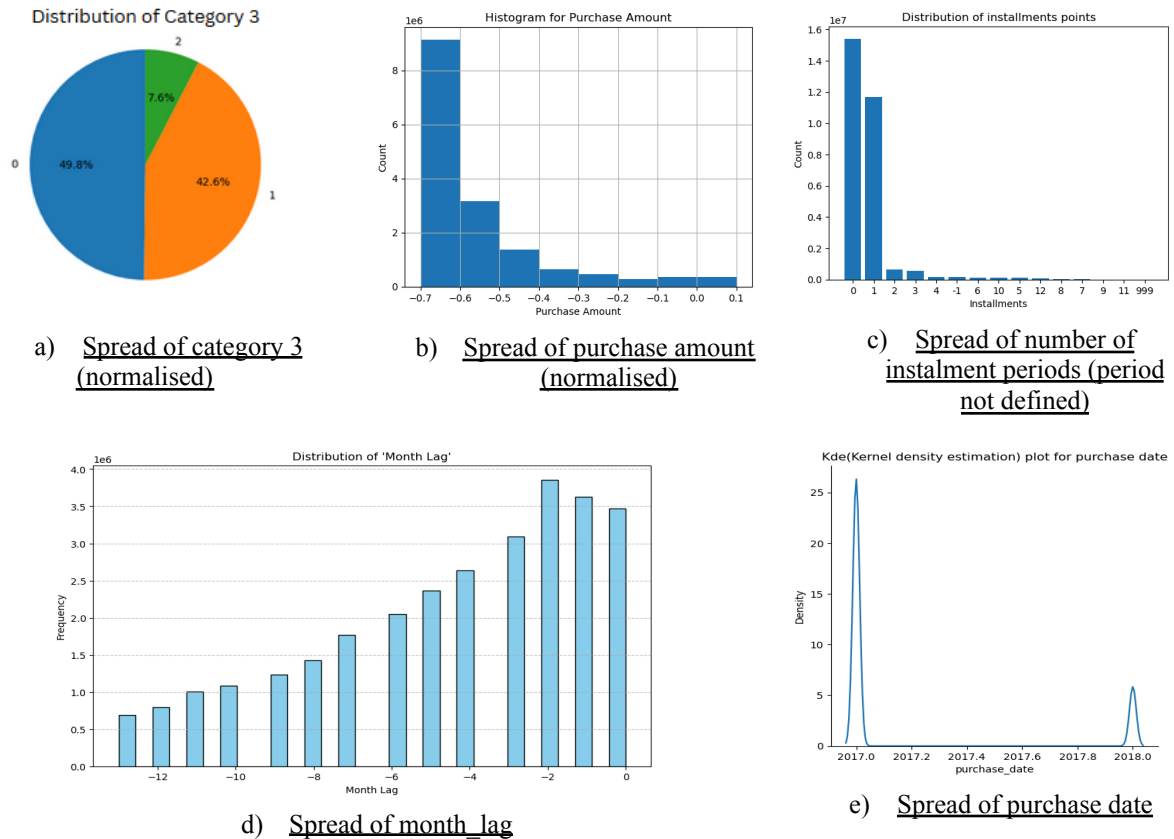
Since card-specific data from train.csv provided little insight into loyalty scores, we had to supplement this data with information about merchant sales (**merchant.csv**) and individual transactions (**historical_transactions.csv** / **new_merchant_transaction.csv**). We first analyse transactional information provided by historical_transactions.csv / new_merchant_transaction.csv, which contained similar features.



Distribution of authorized_flag from historical_transactions (left) and new_merchant_transactions (right)

According to the data definition, authorized_flag of 'N' means the transaction has been voided. Since voided transactions do not contribute to a customer's loyalty, we've removed entries with 'N' flag in our feature engineering.

We then analysed more transactional information regarding purchasing behaviour, such as instalments taken for payment, month lag from the last purchase, purchase date and amount. However, many numerical features are normalised between -1 and 1 and categorical features are anonymised, making their true values and meaning hard to interpret without a reference datum.

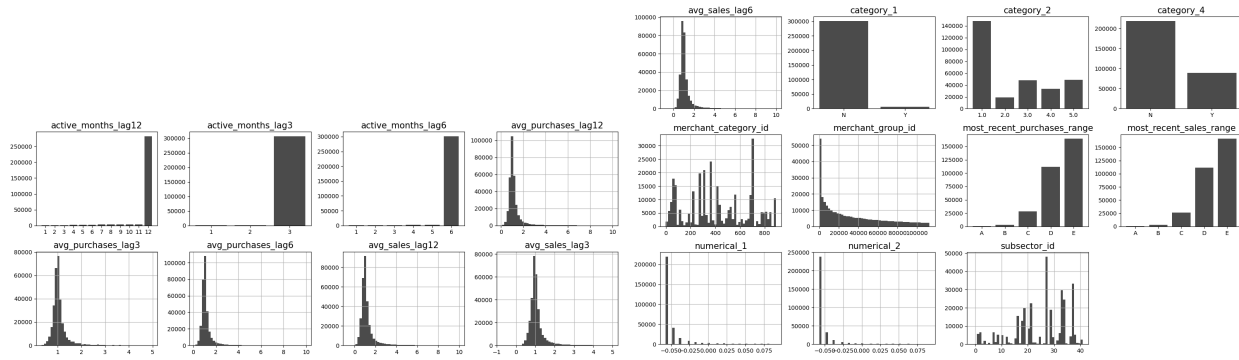


We saw from figure b) that most transactions were made on small purchases. Also, since most small purchases generally do not require instalments, most transactions had little to no instalment periods (figure c). Looking at figure d), most transactions had little to no month lag. We can infer that most transactions were done recently (from the referenced date), due to a variety of reasons (sales, holidays, frequent needs etc). These features could tell us a few things about loyalty score:

- Customers who make small but frequent transactions may have higher loyalty score
- Customers who make occasional to frequent bulk purchases are likely to attain higher loyalty score

From figure e) we saw that most purchases happened between the start of the year. This can also mean that customers who transact more within these periods are also likely to achieve higher loyalty scores.

Finally, we analysed features from merchant.csv, which provided us with merchant-specific information that influences a cardholder's spending pattern. Similarly, many features are normalised or anonymised, making them hard to interpret. Here, a few graphs caught our attention:



From the histogram of **merchant_group_id** we saw that most merchants belong to a small subset of merchant groups, which could focus on essential products and services. This may imply that more transactions are likely to happen here, and customers who spend in these groups frequently are likely to receive higher loyalty scores.

Histograms such as of average sales (**avg_sales***) and average purchases (**avg_purchases***) has shown a positively skewed pattern, implying that most merchants in the dataset belong to the group having low revenue and average transactions. This does not directly relate to a customer's loyalty score however, as loyalty scores depend on their spending habits, such as their amount spent and frequency shopping.

Count plots of **most_recent_sales_range** and **most_recent_purchase_range** revealed that most merchants belong in the range D and E. Since no information is given regarding values A to E, we cannot assume that card holders spending in merchants belonging to these categories may obtain higher loyalty scores.

3. Feature Engineering

3.1. Handling Null Values

Upon loading the dataset we noticed that the following features contain missing data:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 123623 entries, 0 to 123622
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   first_active_month     123622 non-null object
1   card_id                123623 non-null object
2   feature_1              123623 non-null int64
3   feature_2              123623 non-null int64
4   feature_3              123623 non-null int64
```

Non-null count in test.csv

9	avg_sales_lag3	334683	non-null	float64
10	avg_purchases_lag3	334696	non-null	float64
11	active_months_lag3	334696	non-null	int64
12	avg_sales_lag6	334683	non-null	float64
13	avg_purchases_lag6	334696	non-null	float64
14	active_months_lag6	334696	non-null	int64
15	avg_sales_lag12	334683	non-null	float64
16	avg_purchases_lag12	334696	non-null	float64
17	active_months_lag12	334696	non-null	int64
18	category_4	334696	non-null	object
19	city_id	334696	non-null	int64
20	state_id	334696	non-null	int64
21	category_2	322809	non-null	float64

Non-null count in merchants.csv

#	Column	Non-Null Count	Dtype
0	authorized_flag	29112361 non-null	object
1	card_id	29112361 non-null	object
2	installments	29112361 non-null	int64
3	category_3	28934202 non-null	object
4	merchant_id	28973880 non-null	object
5	month_lag	29112361 non-null	int64

Non-null count in historical_transactions.csv

#	Column	Non-Null Count	Dtype
0	authorized_flag	1963031 non-null	object
1	card_id	1963031 non-null	object
2	installments	1963031 non-null	int64
3	category_3	1907109 non-null	object
4	merchant_id	1936815 non-null	object
5	month_lag	1963031 non-null	int64

Non-null count in new_merchant_transactions.csv

We replaced the following null values with the mode of their column.

- 'first_active_month' in **test.csv**
- 'avg_sales_lag3' in **merchants.csv**
- 'avg_sales_lag6' in **merchants.csv**
- 'avg_sales_lag12' in **merchants.csv**

The following null values were replaced as follows, by referring to previous Kaggle solutions

- 'category_2' - 1.0
- 'category_3' - 'A'
- 'merchant_id' - 'M_ID_00a6ca8a8a'

Using these values instead of their mode our RMSE reduced drastically from 3.819 to 3.79 on the baseline decision tree model. Since no further information was provided for these features, we've assumed that domain knowledge was inferred/provided to these Kagglers.

	test_submission.csv	3.72557	3.81967
	Complete (after deadline) · 10d ago		

Submission on Kaggle (Decision Tree) when using mode for all null values

3.2. Splitting Date Variables

5	year_of_first_active	201917 non-null	int32	27	days_since_purchase	27824171 non-null	int64
6	quarter_of_first_active	201917 non-null	int32	28	months_since_purchase	27824171 non-null	float64
7	month_of_first_active	201917 non-null	int32	29	purchase_year	27824171 non-null	int32
8	active_days	201917 non-null	int64	30	purchase_quarter	27824171 non-null	int32
9	active_months	201917 non-null	float64	31	purchase_week_of_month	27824171 non-null	int64
				32	purchase_day_of_week	27824171 non-null	int32
				33	purchase_on_weekend	27824171 non-null	bool
				34	purchase_time_of_day	27824171 non-null	object
				35	purchase_during_holiday	27824171 non-null	bool

Splitting of date fields for 'first_active_month' from train.csv/test.csv (Left) and 'purchase_date' for historical_transactions.csv/new_merchant_transactions.csv

Date variables were divided into year, quarter, month and day (more periodic information such as weekend, holiday, etc if possible) features, to generate more columns that better represent the period of transaction. The number of days elapsed is also calculated to provide trending information that helped in our training.

3.3. Encoding Categorical Variables

When feasible, categorical variables were one-hot encoded into their numerical form to ease data handling by our models. (Certain data operations we used only support numerical variables). Data encoding has also generated more columns to retain as much information as possible after aggregation (see below), which would have led to data truncation if we replaced them with their mode.

3.4. Aggregation

Data was aggregated upon unique card_ids as follows:

Numerical Features	Encoded Categorical Variables	Unencoded Categorical Variables
• sum • mean • min • max • variance • standard deviation • mode	• sum (of the card_id holding that value for that feature)	• nunique (number of unique values in that feature for the card_id) • mode

3.5. Joining Dataframes

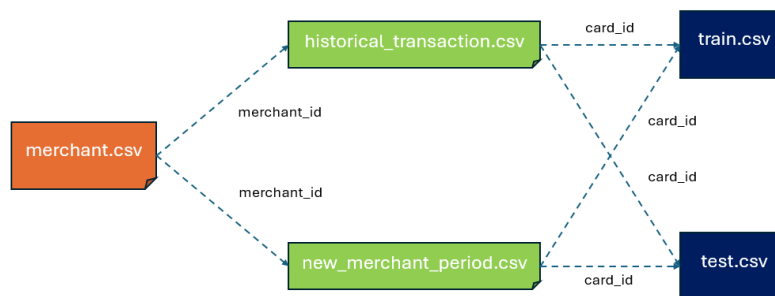


Illustration of dataframe merging

Like what most Kagglers did, we merge the csv files upon the column names above to achieve our final engineered data for our training and test (for generating predictions for submission) datasets. Here, three sets of data were merged upon train.csv and test.csv. Features from historical_transactions.csv (appended ‘_x’), those from new_merchant_transactions.csv (appended ‘_y’) and features from both sets of data combined (containing no ‘_x’ or ‘_y’ affixes). Doing so helped us capture as much variances from each data to generate a more accurate prediction.

4. Models & Findings

4.1. Decision Tree

Decision trees are a type of model mainly used for classification as well as regression problems. In this project, it was used to predict the target variable. We used hyperparameter tuning to enhance the model's accuracy and minimise overfitting. We also discovered the most important features, to predict RMSE using the decision trees model.

4.1.1 Feature Importance

Feature Importances:		
	Feature	Importance
3	active_days	0.742673
9	feature_2_2	0.122708
10	feature_2_3	0.047634
6	feature_1_3	0.030842
5	feature_1_2	0.027607
8	feature_1_5	0.012915
2	month_of_first_active	0.009899
7	feature_1_4	0.005597
0	year_of_first_active	0.000127
1	quarter_of_first_active	0.000000
4	card_id_freq	0.000000
11	feature_3_1	0.000000

Feature Importances (Decision Tree)

We calculated the feature importance of each feature, on the trained decision trees model. The output was a numerical feature, representing its relative contribution to the model, in descending order of importance. We used the most important features to construct the decision trees to get a lower RMSE value. For example, active_days was found to be the most important factor, with an importance of 0.742.

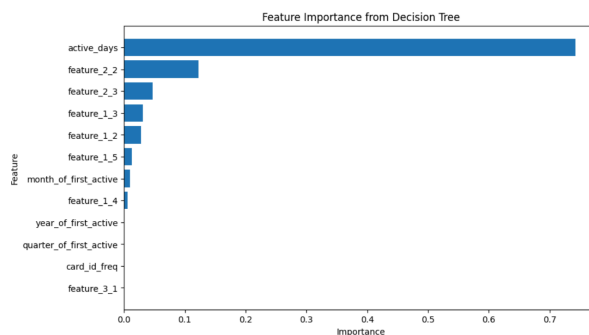


Fig: Horizontal Bar Plot

We constructed a horizontal bar plot to effectively visualise the difference in the importance of different features.

4.1.2. Baseline Model Training

The baseline Decision Trees model is trained using the following hyperparameters:

- Random state: 42
- max_depth: 3 (it controls the maximum depth of the decision tree)
- Criterion: squared_error (minimises squared error)
- Splitter: best (chooses the best split among all features)
- Max_features: none (all are considered for decision trees)

Training on the baseline model gave us an RMSE of 3.78.

4.1.3 K- Fold Validation

In K-fold validation, we are splitting the data into various training as well as validation sets. We used 5-fold validation. The model was trained on 4 folds and then validated on one fold. This was repeated 5 times, to ensure that generalisation occurred. The best-performing hyperparameters were then selected based on the average performance across the 5 folds. This provided us with a better estimate as to how the model would perform on unseen data.

Fitting 5 folds for each of 288 candidates, totalling 1440 fits

Number of fits

4.1.4 Hyper-Parameters Tuning

Hyper-parameters tuning was done using GridSearchCV to get the combination of values with the best score. The search was done using five-fold, giving the best RMSE of 3.76 against our validation set.

Best hyperparameters: {'max_depth': 3, 'max_features': None, 'min_samples_leaf': 20, 'min_samples_split': 2}
Optimized Decision Tree RMSE: 3.763388237987316

RMSE of the optimised decision trees model



dt_submission.csv

Complete (after deadline) · now · Decision Tree only

3.69393

3.79383

Submission on Kaggle (Decision Tree)

Upon submission on Kaggle, it gave an RMSE of 3.79, which gave us insights into the room for improvement, perhaps by using gradient boosting models.

4.2. Random Forest

Random forest is an ensemble learning method primarily used for classification and regression problems. It builds multiple decision trees during training and combines their outputs (average for regression and majority voting for classification) to improve predictive accuracy and control overfitting. Here, Random Forest is used with LightGBM to leverage feature importance and improve prediction accuracy.

4.2.1. Feature Importance

LightGBM is used to add the top 20 and 50 features into the Random Forest model. The feature with the best mean feature importance is popped from the list of features in the data frame and added to a list of best features. Using LightGBM feature importance, we ensured that only the most relevant variables were used for training the Random Forest model.

4.3. Extreme Gradient Boosting (XGBoost)

One of the stronger gradient boosting models, XGBoost incorporates regularisation parameters L1 (Lasso) and L2 (Ridge) to reduce model complexity and control overfitting, a severe problem faced when comparing validation data and Kaggle's public dataset performances. In XGBoost, trees are pruned by predefining their maximum depth instead of using a greedy approach, making it computationally efficient. Splitting of nodes ceases when additional splits do not improve the model's performance. XGBoost's parallel computing capability eased our handling of large datasets and reduced our overall training time.

Using scikit-learn's `train_test_split()` function we've split our feature-engineered data into training and validation sets in an 80:20 ratio. This helps us assess our model's performance on unseen data while preventing data leakage.

4.3.1. Baseline Model Training

Our baseline XGBoost model was trained using the following default hyperparameters:

- `subsample` (Proportion of total data considered): 1.0
- `colsample_bytree` (Proportion of total features considered): 1.0
- `n_estimators` (Number of trees built): 100
- `max_depth` (Depth of each tree): 6
- `learning_rate` (Rate at which model converges to optimal value): 0.1

The baseline model generated an RMSE of 3.84 on our validation set.

4.3.2. HyperParameters Tuning & K-Fold Validation

```
param_grid = {  
    'n_estimators': [50, 100, 150, 200],  
    'max_depth': [4, 5, 6, 7, 9],  
    'learning_rate': [0.01, 0.05, 0.1, 0.2],  
    'subsample': [0.5, 0.6, 0.7, 0.9, 1.0],  
    'colsample_bytree': [0.4, 0.5, 0.7, 0.9, 1.0]
```

Values used for GridSearchCV

To enhance our XGBoost model, we conducted hyper-parameters tuning using GridSearchCV to obtain the combination of values with the best score. GridSearchCV also combines K-FoldCV to better generalise unseen data during the tuning. The search was performed over the following values, on 5, 10, 20 and 30 folds:

The best RMSE obtained from GridSearchCV was 3.704, with the following hyperparameters:

- `subsample`: 0.9
- `colsample_bytree`: 0.4
- `n_estimators`: 100
- `max_depth`: 5
- `learning_rate`: 0.05
- `folds`: 30

4.3.3. Defining Feature Importance

As before, we listed features deemed important by the XGBoost model, in decreasing order of prominence.

	Feature	Importance
55	days_since_purchase_max_y	17563.408203
58	merchant_id_mode	13651.333008
52	avg_sales_lag3_var_y	12901.795898
43	merchant_id_mode_y	9821.165039
47	purchase_amount_max_y	7914.255371
...	...	...
4	month_of_first_active	3.026859
11	feature_2_product	1.840911
18	merchant_id_nunique_x	1.097153
17	installments_mean_x	0.593866
6	feature_sum	0.056695

Important features (XGBoost)

To ensure that only the best features were chosen for training we trained the XGBoost model using a **greedy search strategy** approach. This is done by maintaining a list of best features initialised empty. During each iteration, individual features in the training dataset are trained together with the list of best features. The feature that yields the best RMSE will be added to the list for the next iteration. This process stops when adding features does not improve our model's score.

```
Best selected: days_since_last_purchase_min: 3.7454943656921387
Best selected: month_lag_max: 3.7120752334594727
Best selected: installments_sum: 3.673520088195801
Best selected: month_lag_nunique: 3.6560487747192383
Best selected: purchase_hour_nunique: 3.652273416519165
Best selected: state_id_nunique: 3.6485002040863037
Best selected: purchase_quarter_nunique: 3.643223524093628
Best selected: category_1_N_nunique: 3.6418347358703613
Best selected: category_3_A_nunique: 3.6388602256774902
Best selected: category_3_A_mode: 3.637887716293335
No better features found
Features selected: ['days_since_last_purchase_min', 'month_lag_max', 'installments_sum', 'month_lag_nunique', 'purchase_hour_nunique', 'state_id_nunique', 'purchase_quarter_nunique', 'category_1_N_nunique', 'category_3_A_nunique', 'category_3_A_mode']
```

Features chosen by greedy search strategy

Using the greedy strategy approach we obtained an RMSE of 3.63 on the validation set, with the corresponding best features chosen as shown above. This score increased to 3.728 on Kaggle's public test dataset.

 **xgb_submission.csv**
Complete (after deadline) · now · XGB only

3.64326

3.72852

Submission on Kaggle (XGBoost)

4.4. Light Gradient Boosting Model (LightGBM)

LightGBM is another gradient-boosting ensemble method focused on optimising efficiency. It grows leaf-wise, unlike the traditional trees that use depth-wise growth, which means it grows the leaf with the biggest loss reduction, creating an asymmetric tree that captures complex patterns more efficiently. With the efficiency of the LightGBM model, it is possible to add Histogram-based learning, which buckets continuous features into discrete bins. This significantly speeds up the computation and reduces memory usage, hence allowing LightGBM to handle large and high-dimensional datasets efficiently. LightGBM offers regularisation parameters that allow users to prevent overfitting and enhance generalisation.

As before we split our dataset into Train and Validation sets in an **80:20** ratio, ensuring unbiased evaluation. We then proceeded to perform hyperparameter tuning and cross-validation to optimise the model on the Train set (80%), and the final model was tested on the Validation set (20%) to assess the model performance.

4.4.1. Baseline Model to extract top 200 important features

	Feature	Importance
17	merchant_id_mode_x	76
459	merchant_id_mode	75
239	merchant_id_mode_y	60
576	days_since_purchase_min	33
251	purchase_amount_max_y	31
...	...	...
20	month_lag_min_x	1
280	avg_sales_lag3_min_y	1
89	avg_purchases_lag6_mean_x	1
129	city_id_mode_x	1
128	city_id_nunique_x	1

200 rows x 2 columns

With 671 features available, we decided to use a baseline model (mostly default parameters, but with 'feature_fraction': 0.8, 'bagging_fraction': 0.8,) to extract the top 200 most important features and train upon that.

The baseline model gave an RMSE of **3.7352**, with the 200 most important features as shown on the left.

Important features (LightGBM with slight tuning in feature_fraction and bagging_fraction)

4.4.2. Hyperparameters Tuning

To optimise the model performance, such that we can achieve the lowest RMSE, we used Optuna for the hyperparameter tuning. Optuna is an open source that automates the search for the best parameters. Unlike traditional grid or random search methods, it uses a more efficient and adaptive Bayesian optimisation, finding the best combination of parameters that result in the lowest RMSE without overfitting the model.

Below are the key parameters considered in the tuning process:

learning_rate (Controls the speed at which the model learns):

num_leaves (Maximum tree leaves for base learners)
max_depth (Maximum tree depth for base learners)
feature_fraction (% of randomly selected features at the beginning of tree construction)
bagging_fraction (checks for the data fraction to be utilised in each iteration)
lambda_l1 (L1 regularisation)
lambda_l2 (L2 regularisation)
bagging_freq (bagging_freq = k means perform bagging at every k iteration)

To ensure our LightGBM model is robust, we used **K-Fold cross-validation** on the important features. This helps to reduce bias in the data by validating the model across multiple folds of the data. We employed RepeatedKfold, with 5 folds and 2 repeats to reduce the variance to a greater extent and endure the stability of the result across different splits.

Optuna explored ranges as such:

```
params = {
    'objective': 'regression',
    'metric': 'rmse',
    'boosting_type': 'gbdt',
    'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1, log=True),
    'num_leaves': trial.suggest_int('num_leaves', 20, 50),
    'max_depth': trial.suggest_int('max_depth', -1, 15),
    'min_data_in_leaf': trial.suggest_int('min_data_in_leaf', 15, 30),
    'feature_fraction': trial.suggest_float('feature_fraction', 0.5, 0.9),
    'bagging_fraction': trial.suggest_float('bagging_fraction', 0.6, 0.9),
    'bagging_freq': trial.suggest_int('bagging_freq', 1, 10),
    'lambda_l1': trial.suggest_float('lambda_l1', 0.0, 10.0),
    'lambda_l2': trial.suggest_float('lambda_l2', 0.0, 10.0),
    'verbose': -1
}
```

Hyper-Parameter Tuning Using Optuna

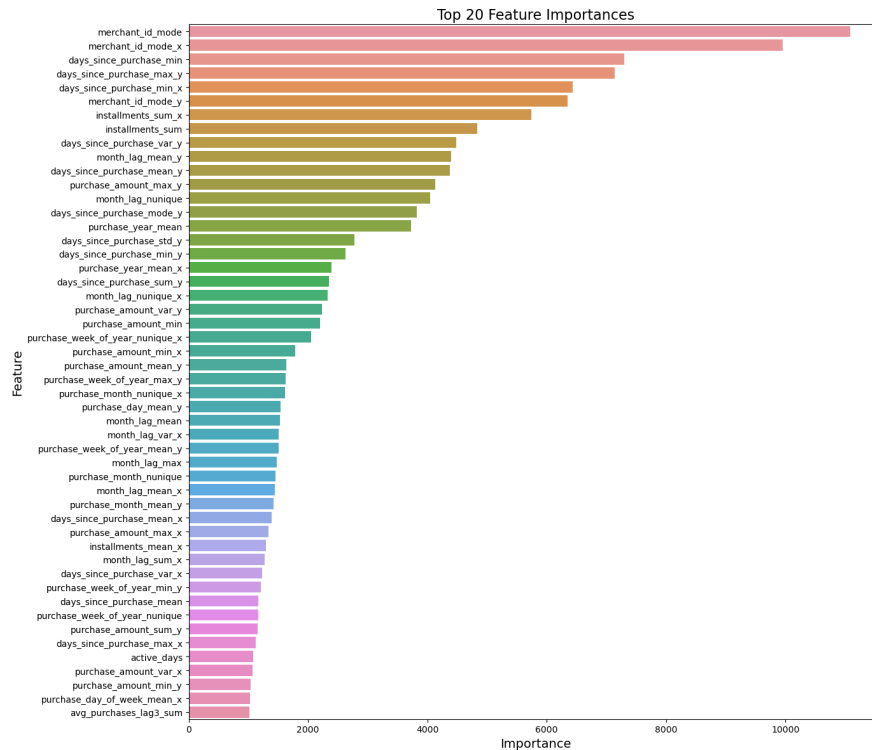
The best parameters were:

learning_rate: 0.011297128820380701
num_leaves: 45
max_depth: 15
min_data_in_leaf: 27
feature_fraction: 0.518 (3s.f.)
bagging_fraction: 0.622 (3s.f.)
lambda_l1: 7.93 (3s.f.)
lambda_l2: 3.56 (3s.f.)
bagging_freq: 3

The best RMSE obtained was an average of **3.68** (3s.f.) across the validation sets using K-Fold validation

4.4.3. Feature Importance

We trained the model with the top 200 important features and the best parameters obtained. The model train was tested on the validation set, giving an RMSE of 3.717(4s.f.).



Final Top 20 Important Features (LightGBM)



lgb_submission.csv

Complete (after deadline) · 41m ago · LightGBM only

3.64782

3.73013

Submission on Kaggle (LightGBM)

After submitting it on Kaggle, we obtained a public score of **3.730** (4s.f.).

4.5. CatBoost

CatBoost, short for Categorical Boosting, is a powerful gradient boosting library developed by Yandex, tailored for efficient handling of categorical data. CatBoost uses symmetric trees, where splits are made based on the same feature for all nodes at a given depth. This approach speeds up the training process and reduces memory consumption, making CatBoost highly efficient, even for large datasets. It internally converts categorical features into numerical representations using advanced techniques like ordered boosting and target statistics, eliminating the need for extensive preprocessing such as one-hot encoding.

Ordered boosting, a distinctive feature of CatBoost, mitigates overfitting by employing dataset permutations during tree construction, effectively preventing target leakage. This innovative approach ensures robust and accurate model performance, particularly in scenarios involving categorical variables.

In the implementation of CatBoost, categorical features must be specifically defined when fitting the model. Non-numerical data in the dataset is converted to categorical data in our data preprocessing. After which, a variable `cat_features` that consists of all data with “category” data type is passed into the CatBoost model as one of the parameters of the model. From there, as mentioned above, the CatBoost model will automatically process these categorical features by converting them into numeric representations using ordered boosting and target statistics.

```
# Initialize CatBoost Regressor
catboost_model = CatBoostRegressor(
    iterations=1000, # Number of boosting iterations
    learning_rate=0.05, # Learning rate
    depth=8, # Depth of trees
    eval_metric='RMSE', # Evaluation metric
    cat_features=categorical_features, # Specify categorical features
    verbose=100 # Prints progress every 100 iterations
)
```

Parameters used for CatBoost base model training

Due to the advantages of CatBoost, designed to have reduced preprocessing time, better performance with less hyperparameter tuning, and robustness to overfitting, the runtime of the CatBoost model was quick, taking only 19 minutes 8 seconds to complete the training of the model. The in-built overfitting detector helped to prevent overfitting and stopped at iteration 524 after finding the best score at iteration 523.

The train dataset is split into a train and validation set with a factor of 0.2.

```
0:   learn: 3.8597615   test: 3.7609718 best: 3.7609718 (0)   total: 838ms   remaining: 13m 57s
50:   learn: 3.6920452   test: 3.6625273 best: 3.6625273 (50)   total: 1m 3s   remaining: 19m 43s
100:  learn: 3.6650410   test: 3.6584708 best: 3.6584708 (100)   total: 2m 18s   remaining: 20m 36s
150:  learn: 3.6508023   test: 3.6568112 best: 3.6568058 (144)   total: 3m 39s   remaining: 20m 31s
200:  learn: 3.6362097   test: 3.6552016 best: 3.6551985 (198)   total: 5m       remaining: 19m 56s
250:  learn: 3.6232929   test: 3.6537995 best: 3.6537821 (249)   total: 6m 37s   remaining: 19m 47s
300:  learn: 3.6138209   test: 3.6528698 best: 3.6528657 (295)   total: 8m 17s   remaining: 19m 15s
350:  learn: 3.6031606   test: 3.6526841 best: 3.6526489 (331)   total: 10m 7s   remaining: 18m 43s
400:  learn: 3.5944117   test: 3.6522361 best: 3.6522203 (396)   total: 11m 56s   remaining: 17m 50s
450:  learn: 3.5852550   test: 3.6517064 best: 3.6517064 (450)   total: 13m 44s   remaining: 16m 43s
500:  learn: 3.5764812   test: 3.6511629 best: 3.6511388 (492)   total: 15m 26s   remaining: 15m 22s
550:  learn: 3.5672061   test: 3.6506370 best: 3.6504266 (523)   total: 17m 17s   remaining: 14m 5s
600:  learn: 3.5608024   test: 3.6506688 best: 3.6504266 (523)   total: 19m 8s   remaining: 12m 42s
Stopped by overfitting detector (100 iterations wait)

bestTest = 3.650426627
bestIteration = 523
```

Results of the training of the model on the train dataset.

The best RMSE score from training the CatBoost model is 3.650 (4s.f.).

	cat_submission.csv	3.64057	3.73086
	Complete (after deadline) · 4m ago · CatBoost only		

Submission on Kaggle (CatBoost)

After submitting it on Kaggle, we obtained a public score of 3.730 (4s.f.). The score is likely to be higher than the trained score due to other outliers from the Kaggle test set.

5. Ensemble Learning (Stacking different Models)

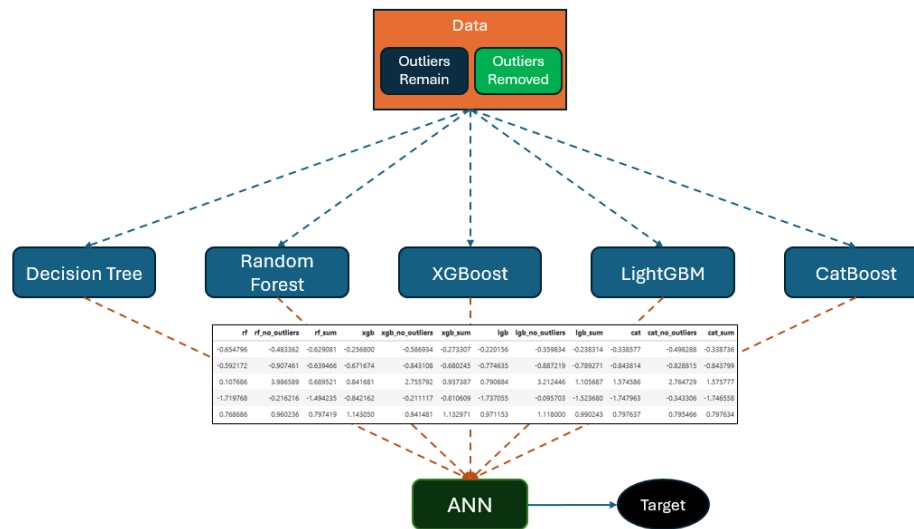


Illustration of ensemble learning

Each base model was retrained with two types of datasets, one with outliers removed and the other remaining, generating two predictions each (The prediction with outliers removed received a better RMSE of 1.56 on average). Weighted summing is then applied across the predictions to obtain an aggregated prediction. Predictions with outliers were assigned a higher weight (80%) than those with outliers removed (20%) to prevent the meta-model from becoming biased towards data without outliers. In total, 11 predictions (two for each model + the prediction from weighted summing) were fed into an Artificial Neural Network meta-learner (see below) to combine weak learners and train a stronger overall model.

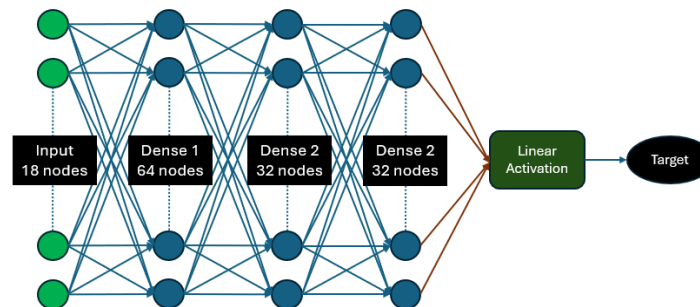


Illustration of the neural network used. Each dense layer consists of a Rectified Linear Unit (ReLU) at its output

Dropout layers (10% of each layer's output) were inserted between the dense layers to reduce overfitting. The number of nodes for each layer and dropout ratios were determined through trial-and-error testing. The model is then trained over 50 epochs (With early stopping patience set at 10 to stop training if scores do not improve for 10 rounds). Stacking the base models with this basic neural network yielded a score of 3.695 (4.s.f.) on our validation set and **3.713** (4.s.f.) on Kaggle's public dataset.

```

Epoch 10/50
505/505 ————— 2s 4ms/step - loss: 13.8965
80/253 ————— 0s 1ms/step

C:\Users\SQ\anaconda3\Lib\site-packages\keras\src\ops\nn.
s has size 1. The softmax operation will always return th
warnings.warn(

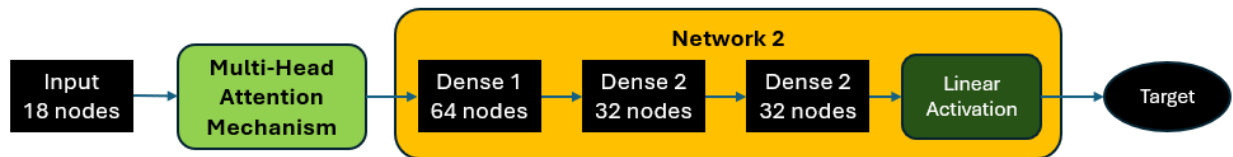
253/253 ————— 1s 2ms/step
RMSE on the val set: 3.69563601693899

```

Score on validation set (Basic Neural Network)

	nn_submission.csv Complete (after deadline) · 8m ago	3.63573	3.71378
--	--	----------------	----------------

Submission on Kaggle (Basic Neural Network)



Neural Network with Multi-Head Attention Mechanism

We refined the Neural Network further by using a multi-head attention mechanism layer, which focuses on features with higher importance during training. This layer is made up of three dense layers as follows:

- **Queries (Q):** What the model is searching for.
- **Keys (K):** Potential matches.
- **Values (V):** Actual information associated with the keys.

With the multi-head attention layer, our score improved to 3.675 (4s.f) on our validation set and **3.696** (4.s.f.) on Kaggle’s public dataset.

```

Epoch 10/50
505/505 ————— 2s 4ms/step - loss: 14.2722
89/253 ————— 0s 1ms/step

C:\Users\SQ\anaconda3\Lib\site-packages\keras\src\ops\nn.
s has size 1. The softmax operation will always return th
warnings.warn(

253/253 ————— 1s 2ms/step
RMSE on the val set: 3.675051140627789

```

Score on validation set (Neural Network with Multi-Head Attention Mechanism)

	test_submission.csv Complete (after deadline) · 5m ago · MT6.1	3.61376	3.69610
--	--	----------------	----------------

Submission on Kaggle (Neural Network with Multi-Head Attention Mechanism)

As final enhancements, we’ve trained with 50 separate neural network models, each with a randomly assigned number of epochs (in multiples of 10) to introduce variance between them. The predictions were averaged to derive our final prediction, minimising the effect of noise

picked up by each meta model. Our score improved to 3.649 (4s.f) on the validation set and **3.683** (4s.f) on Kaggle, which commensurates to 337th place out of 4111 submissions in ELO's competition ranking.

```
Epoch 50/50
505/505 ————— 2s 4ms/step - loss: 13.8825
83/253 ————— 0s 1ms/step

C:\Users\SQ\anaconda3\Lib\site-packages\keras\src\ops\nn.
s has size 1. The softmax operation will always return th
warnings.warn(

253/253 ————— 1s 2ms/step
RMSE on the val set: 3.649906228024103
```

Score on validation set (Average of 50 Neural Networks)

 nn_submission.csv	3.60705	3.68370
Complete (after deadline) · now		

Submission on Kaggle (Average of 50 Neural Networks)

336	hiromu		3.68369	115	6y
337	jenjen		3.68370	8	6y
338	konradcieslik		3.68372	40	6y

Ranking on Kaggle

6. Challenges

Our dataset contained many ambiguous features, such as anonymised categories (category_1, category_2, feature_1, feature_2, etc) and normalised numerical variables (target, purchase_amount, etc). With target values consisting of outliers below -30, removing them has caused massive overfitting on our training dataset and large generalisation error on Kaggle's public dataset, prompting us to retrain our base models on data with and without outliers and apply a weighted sum on them. Weak correlations between features and our prediction target has encouraged us to perform extensive feature engineering. Nevertheless, we conducted multiple trials and errors to produce the most optimal result from our feature-engineered data.

So why hadn't we surpassed the scores of 7th-place (RMSE 3.64108) and 10th-place (RMSE 3.64604)? We felt that more could be explored into feature engineering to generate more features that better represent our data, such as predicting intermediary features for machine learning. Also, our neural network meta-learner may not be the most optimal, as better neural layers can be implemented to augment the model. We felt that these could be done if we had the benefit of time.

7. Takeaways

Criticality of feature engineering

We've learnt to be critical when preprocessing our data, ensuring that our models learn them effectively. For instance, encoding categorical data helps reduce information loss when transactions are aggregated upon individual card ids. Splitting date variables helped us capture more trend information that can be learnt by our models. Finally, the handling of null values impacted our prediction accuracy greatly, as seen from using specific values from domain knowledge to replace null entries of 'category_2', 'category_3' and 'merchant_id' instead of their mode.

Severity of overfitting

We saw across our models that despite achieving a low RMSE in our validation set, the score from Kaggle's public dataset remains substantially higher, likely due to the increased presence of outliers. We've addressed it by employing various techniques, such as:

- Using **regularisation parameters L1 and L2** to reduce model complexity
- **Bagging, k-Fold** cross-validation by averaging predictions across different subsets of data, reducing the impact of noise
- **Dropout** layers in NN meta-learner to reduce the chances of learning noises in our data

Importance of ensemble learning

We saw that the neural network meta learner improved our accuracy by combining the strengths and reducing the errors of each base model, bringing our RMSE below 3.70. This could not be achieved by individual models, regardless of how their hyperparameters were tuned. We've learnt that for machine learning problems that are error-sensitive, stacking models and leveraging each of their advantages can be more effective than comparing individual model performances and selecting the best one.